

מקור אמת אחד

שבוע: 9 ניקוד: 100 מועד הגשה: לפני תחילת שיעור 10

משקל בציר הסופי: מטלת כיתה 9 · חלק ב · שבעים אחוז מציון המטלה · 10 הטובות מתוך 12 נספח **מסמך** שיעור: 60 דקות

מטרה

לקחת יישום עובד ולשנות את **הצורה** שלו, בלי לשנות אף דבר במה שהוא עושה — עד שארבע תקלות שיש בו היום הופכות לבלתי אפשריות.

היום אין פיצ'ר חדש. יש קובץ אחד באורך מאתיים שורות, שעובד, ושיש בו ארבע התנהגויות שגויות שאי אפשר לתקן בטלאי. בסוף השיעור הוא יהיה ארבעה קבצים, ארבע התקלות ייעלמו מבלי שאף אחת מהן תתוקן ישירות, וזו תהיה **הצורה שכל פרויקט בקורס בנוי בה ושמתחיל שבוע 13 כתוב מולה**.

זו לא המלצה. הפרויקט שלך חייב להיות בנוי כך, וגם הבודק האוטומטי וגם ה-CI במאגר שלך בודקים את זה.

תוצרי למידה

- לזהות **שני מקורות אמת** ביישום, ולהסביר איזו התנהגות שגויה הם מייצרים.
- לפרק קובץ אחד לשלוש שכבות — נתונים, ציור, אירועים — ולדעת מה אסור לכל אחת.
- לכתוב `render(state)` שהוא פונקציה של הארגומנט שלו ותו לא.
- לכתוב מטפל אירועים שרק **מתאר מה נכון עכשיו**, ולא מצייר כלום.
- לגזור** תצוגה — חיפוש, סינון, מיון, מונים, סיכומים — במקום לשמור אותה.
- להשתמש ב**מזהה יציב** לכל פריט, ולהסביר למה אינדקס במערך אינו זהות.

מה יש בתיקייה

קובץ	מה עושים איתו
js/app.js	הקובץ שמפרקים . נתון, עובד, מאתיים שורות
js/items.js	הנתונים. מחליפים בפרטים של הישות שלך
js/state.js	ריק כמעט. שכבה 1 — האמת
js/render.js	ריק כמעט. שכבה 2 — הציור
js/events.js	ריק כמעט. שכבה 3 — האירועים
js/summary.js	נתון ועובד. חלק ג מוחק אותו
observations-he.md	חלק א. גיליון התצפיות. ממלאים לפני שמתקנים
index.html	נתון ומלא. לא נוגעים בו
summary.html · item.html	נתונים. שורה אחת בכל אחד משתנה בחלק ג

מריצים עם Live Server, תמיד. היישום בנוי ממודולים, ומודול לא נטען מעל `file://`.

אין `if` שמתקן תקלה. ארבע התקלות אמורות להיעלם כתוצאה מהפירוק. תיקון נקודתי של אחת מהן במקום שבו היא נמצאת היום הוא התשובה של שבוע 8, והיא זו שייצרה אותן.

`js/state.js` לא נוגע ב-DOM. אין בו `document`, אין `querySelector`, אין `textContent`. חפש את המילה `document` בקובץ הזה כשסיימת – אם היא שם, משהו זלג.

`js/render.js` לא משנה state ולא רושם מאזינים. מאזין שנרשם בתוך `render` נרשם מחדש בכל שינוי.

`js/events.js` לא מצייר. אין בו `createElement`, `append`, `remove()`, `hidden` או `classList`. יש בו `setState` וזהו.

מזהים, לא אינדקסים. `data-id` על כל שורה. `data-index` הוא אפס נקודות בחלק א.

ההתנהגות והמראה לא משתנים. זה ריפקטור. אם המסך נראה אחרת בסוף – חוץ מארבע התקלות שנעלמו ומהפקדים שלא עבדו – שינית יותר מדי.

חלק א · הפירוק · 40 נקודות ליבה

א.1 · ארבע התצפיות · 8 נקודות ליבה

לפני שאתה כותב שורה. פתח את `index.html` ובצע את ארבעת הרצפים שכתובים בראש `js/app.js` ובגיליון `observations-he.md`. מלא את הגיליון: מה ראית, מה היה אמור לקרות, איפה בקוד, ואיזה משלושת החוקים נשבר.

נקודות ניתנות על תצפית, לא על מסקנה. "המסך הראה כמון טחון: 3 ואחרי ניקוי החיפוש קמח מלא היה 3 ו כמון טחון היה 2" שווה את מלוא הנקודות גם אם השורה האחרונה בבלוק שגויה. "יש באג באינדקסים" בלי מה שראית שווה אפס.

א.2 · שלוש השכבות · 32 נקודות ליבה

העבר את הקוד מ- `js/app.js` לשלושת הקבצים. מעבירים, לא כותבים מחדש – רוב השורות עוברות כמו שהן, וכל שורה שהשתנתה היא שורה שאתה צריך להיות מסוגל להסביר.

שכבה	הקובץ	מה יש בו	מה אסור לו
1	<code>js/state.js</code>	אובייקט <code>state</code> אחד, <code>setState</code> , וכל הגזירות	לגעת ב-DOM
2	<code>js/render.js</code>	<code>render(state)</code> וכל בוני האלמנטים	לשנות <code>state</code> · לרשום מאזינים · לקרוא את המסך
3	<code>js/events.js</code>	<code>wire()</code> וכל המאזינים	לצייר

`js/app.js` נשאר עם ארבע שורות עבודה, והן נתונות לך בפתרון של השבוע הבא. הסדר חשוב:

#	השורה	למה בסדר הזה
1	הכנס את נתוני הזריעה <code>state-7</code>	לפני שמישהו מצייר
2	<code>subscribe(render)</code>	המסך עוקב אחרי כל שינוי, מעכשיו ולתמיד

#	השורה	למה בסדר הזה
3	<code>wire()</code>	פעם אחת, ולא מתוך <code>render</code>
4	<code>render(getState())</code>	זו השורה ששוכחים. ההרשמה לא מפעילה כלום — היא נרשמת לשינויים הבאים , ועדיין לא היה שינוי. בלעדיה העמוד ריק עד שהמשתמש נוגע במשהו, וזה נראה בדיוק כמו <code>render</code> שבו

שתי דרישות שהן חלק מהפירוק ולא תיקון:

• **מזהה יציב.** כל `<li>` נושא `data-id` עם המזהה של הפריט. המיקום של שורה משתנה בכל מיון וחיפוש; הזהות שלה לא.

• **מאזין אחד על המכל.** `render` מרוקן את `#list` בכל שינוי, ולכן מאזין שנרשם על שורה מת איתה. עד השבוע הזה רינדור מחדש היה בחירה; מהשבוע הזה הוא קורה בכל הקשה.

חלק ב · מה שאפשר לגזור – נגזר · 25 נקודות ליבה

שלושת הפקדים בעמוד — חיפוש, מדף, מיון — **אינם שלוש התנהגויות**. הם שלושה שדות ב-`state`, ופונקציה אחת שמחשבת מהם מה מוצג.

1. `selectVisible(state)` — מסנן לפי `state.query`, מסנן לפי `state.category`, ואז ממין לפי `state.sort`. מחזיר מערך חדש.
2. `selectCategories(state)` — שמות המדפים שקיימים באוסף, בלי כפילויות, ממוינים. `render` בונה מהם את ה-`<option>` ים.
3. שלושת המטפלים באירועים הם **שורה אחת כל אחד**. מטפל מיון שאורכו יותר משורה אחת ממין משהו, ומיון הוא גזירה.

מה נגזר	ואיפה	ולמה לא נשמר
הרשימה המוצגת	<code>selectVisible</code>	מערך שמור לצד <code>items</code> מתיישן ברגע ש- <code>items</code> משתנה
<code>#count</code>	<code>render</code> , מאורך המערך המוצג	מונה שמישהו מעדכן הוא מונה שמישהו ישכח
רשימת המדפים	<code>selectCategories</code>	הסר את הפריט האחרון במדף — האפשרות נעלמת בלי שאף אחד יכתוב שורה
שני המצבים הריקים	<code>render</code>	"אין כאן כלום" ו"החיפוש לא מצא" הם שני מצבים ורק אחד מהם שולח לטופס

המיון משווה מחרוזות

`a.title < b.title` משווה קודי תווים, וזה אינו סדר אלפביתי לא בעברית ולא באנגלית.
`a.title.localeCompare(b.title, 'he')` הוא הסדר שאדם מצפה לו.
ומיינים **עותק**. `state.items.filter(...)` כבר מחזיר מערך חדש, ולכן ה-`sort` שאחריו בטוח. מיון ישיר של `state.items` הוא פקד תצוגה שעורך את הנתונים.

חלק ג · שלושה עמודים, render אחד · 20 נקודות הרחבה

`js/app.js` לא יודע באיזה עמוד הוא נמצא, ולא צריך לדעת: כל אזור ב- `render` חוזר מיד אם נקודת העיגון שלו אינה קיימת. לכן הוספת עמוד היא הוספת עוגן ופונקציה, ואפס שינויים במקום אחר.

- עמוד הסיכום.** שנה שורה אחת ב- `summary.html` – הסקריפט הוא `js/app.js` – **ומחק את `js/summary.js`**.
`selectSummary(state)` מחזיר את המספרים ואת הפילוח לפי מדף, ו- `render` כותב אותם. מחיקת קובץ היא חלק מהריפקטור, לא ניקיון אחריו.
- עמוד הפריט.** הוסף שורה אחת ל- `item.html` – אותו סקריפט – ו- `render` יחליף את מה שיש שם היום, שהוא **תמונה** של פריט: סימון נכון שאינו נכון על אף פריט. הפריט נבחר לפי `?id=` מה-URL, דרך `selectOne(state, id)`.
- מזהה שאינו קיים הוא מצב ריק,** לא שגיאה ולא עמוד לבן: משפט שאומר מה קרה ודרך חזרה. פתח את `item.html?id=nope` ובדוק.
- כל שורה ברשימה מקשרת לעמוד הפריט שלה.

מה זה מוכיח

העמוד השלישי מציג את אותם נתונים בדיוק, ואין דרך שבה הוא יסתור את עמוד הרשימה. לא משום שמישהו משאיר אותם מסונכרנים – אלא משום שיש **אחד**. זה כל הרווח מהיום, במקום אחד שאפשר להצביע עליו.

חלק ד · ביטול הפעולה האחרונה · 10 נקודות אתגר

הסרה מציעה **בטל**. לחיצה על "בטל" מחזירה את המצב לקדמותו.

מותר לך שדה אחד חדש ב- `state`, והוא לא נגזר – הוא זיכרון של מצב שכבר לא קיים, וזה בדיוק הסוג של דבר שכן שייך ל-`state`. שמור את **המערך שהחלפת**, שלם, ולא תיאור של מה שהוסר. הביטול הוא אז השמה אחת.

מה	איפה
השדה החדש	<code>state</code>
ההצעה על המסך	<code>render</code> , מתוך אותו שדה
השמירה והשחזור	<code>events</code> , שתי קריאות ל- <code>setState</code>

אם הביטול שלך ארוך מארבע שורות, אתה משחזר במקום להחזיר.

למה בטל ולא "אתה בטוח?"

אישור גובה מכל משתמש את מחיר הטעויות של המעטים, ואחרי שבוע לוחצים עליו בלי לקרוא. ביטול לא עולה כלום מראש והוא מכסה גם את המחיקה שהמשתמש התכוון אליה והתחרט עליה עשר שניות אחר כך. הכלל: מאשרים רק את מה שאי אפשר לבטל.

מה	כמה
אין <code>var</code> · אין <code>innerHTML</code> עם טקסט שמשתמש הקליד	2
היגיינת קומיטים: קומיט לכל חלק, הודעה שאומרת מה השתנה	3

הריפקטור הזה הוא המקום הטוב ביותר בסמסטר לקומיטים קטנים. קומיט אחד ענק בשם `refactor` הוא קומיט שאי אפשר לחזור ממנו לאמצע.

מדיניות AI לשבוע הזה

שותף מימוש, כמו משבוע 7 — עם אזהרה שספציפית לשבוע הזה.

ריפקטור הוא בדיוק המשימה שבה Claude Code נראה הכי טוב והכי מסוכן: הוא יעביר את הקוד לשלושה קבצים בשלושים שניות, והוא **יתקן תוך כדי** דברים שלא ביקשת. תיקון שלא ביקשת הוא שינוי התנהגות בתוך ריפקטור, וזו הדרך הבטוחה ביותר להגיע ליישום שאתה כבר לא מסוגל להסביר.

שתי טעויות שהוא עושה כאן שוב ושוב, ושתיהן עוברות בדיקה ידנית:

1. הוא שומר שדה נגזר ב- `state` — `categories`, `count`, `visibleItems` — כי זה מהיר יותר. זה גם בדיוק הבאג שהיום נועד למחוק.

2. הוא רושם מאזין בתוך `render`, כי שם נוח למצוא את האלמנט.

קרא את הפלט מול שלוש השורות בטבלה של חלק א.2 לפני שאתה מריץ אותו. שתי הטעויות האלה עולות נקודות, ובמבחן הן עולות זמן.

הגשה

תיקיה `week-09/` במאגר הסמסטר שלך, ובה כל מה שהיה בתיקיית המטלה — כולל `observations-he.md` המלא.

הגשה במודל: כתובת המאגר + ה-SHA של הקומיט. ה-SHA מקפיא את מה שנבדק.

אחרי ההגשה, העתק את `js/state.js`, `js/render.js`, `js/events.js` ו-`js/app.js` גם ל-`project/` שלך. **מכאן והלאה הפרויקט בנוי כך, ומשבוע 10 מוסיפים לו שכבות — לא משנים את השלוש האלה.**

הגשה מאוחרת: ההגשה נסגרת במועד שכתוב במודל. הגשה שמגיעה אחרי המועד מתקבלת ונבדקת במלואה, ומסומנת כמאוחרת — היא אינה מורידה נקודות.

סיימת כשכל אלה נכונים

☐ `observations-he.md` מלא — ארבעה בלוקים, ובכל אחד שורת "מה ראיתי" שמתארת מסך.

☐ בקובץ `js/state.js` אין את המילה `document`.

☐ בקובץ `js/events.js` אין `createElement`, `textContent`, `append` או `hidden`.

- ☐. `js/app.js` קצר מעשרים שורות קוד.
- ☐. הסרת פריט, ואז הוספת פריט — **הפריט שהוסר לא חוזר**.
- ☐. חיפוש, ואז הוספת פריט שאינו מתאים לחיפוש — הוא **לא** מופיע. ניקוי החיפוש — הוא כן.
- ☐. חיפוש, `+1` על שורה, ניקוי החיפוש — **אותה שורה** עלתה.
- ☐. מיון לפי כמות, ואז הוספה — הפריט החדש נכנס **במקום הנכון**.
- ☐. בחירת מדף ב- `#category` מסננת, והרשימה שלו נבנתה מהנתונים.
- ☐. רוקנת את `js/items.js` וראית את המצב הריק ואת `0`; החזרת אותו וראית שהכול חזר.
- ☐. פתחת את `item.html?id=nope` וקיבלת משפט ודרך חזרה.